

Top 36 LLM Interview Questions and Answers for 2026

This article provides a comprehensive guide to large language model (LLM) interview questions, covering fundamental concepts, intermediate and advanced techniques, and specific questions for prompt engineers.

Updated Jan 15, 2026 · 15 min read

[Large language models](#) (LLMs) have become increasingly important in artificial intelligence, with applications across various industries.

As the demand for professionals with LLM expertise grows, this article provides a comprehensive set of interview questions and answers, covering fundamental concepts, advanced techniques, and practical applications.

If you're preparing for a job interview or simply want to expand your knowledge, this article will be useful.

Basic LLM Interview Questions

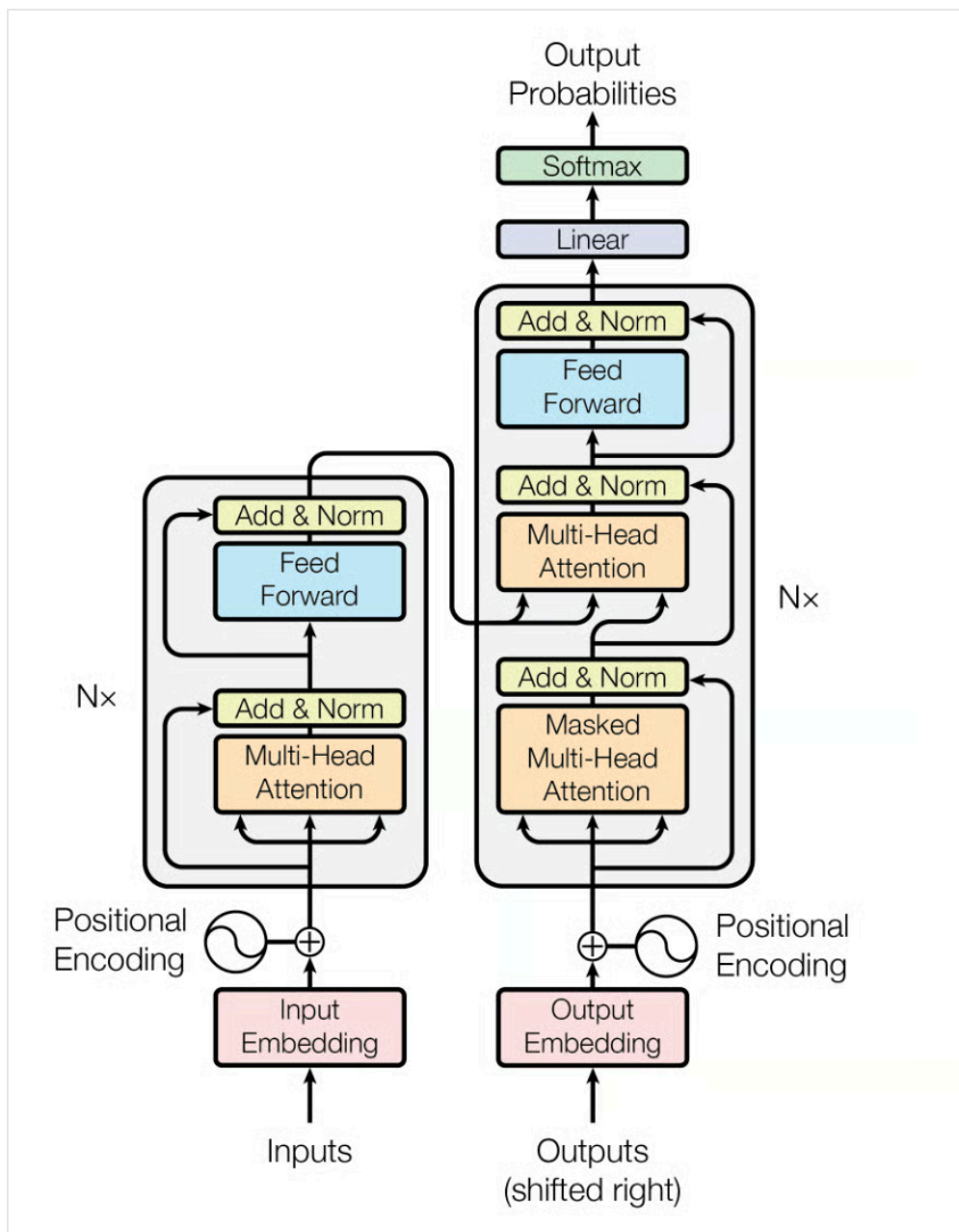
To understand LLMs, it's important to start with the fundamental concepts. These foundational questions cover essential aspects such as architecture, key mechanisms, and typical challenges, providing a solid base for learning more advanced topics.

What is the Transformer architecture, and how is it used in LLMs?

The [Transformer architecture](#) is a deep learning model introduced by [Vaswani et al.](#) in 2017, designed to handle sequential data with improved efficiency and performance than previous models like recurrent neural networks (RNNs) and long short-term memory (LSTMs).

It relies on [self-attention mechanisms](#) to process input data in parallel, making it highly scalable and capable of capturing long-range dependencies.

In LLMs, the Transformer architecture forms the backbone, enabling models to process large amounts of text data efficiently and generate contextually relevant and coherent text outputs.



The Transformer model architecture. [Source](#)

Explain the concept of "context window" in LLMs and its significance.

The **context window in LLMs** refers to the range of text (in terms of tokens or words) that the model can consider at once when generating or understanding language. The significance of the context window lies in its impact on the model's ability to generate logical and relevant responses.

Generally, a larger context window allows the model to consider more context, leading to better understanding and text generation, especially in complex or lengthy conversations. However, it also increases computational requirements, making it a balance between performance and efficiency.

Additionally, recent research reveals that most models show degraded performance well before their advertised limits. Models can exhibit a "lost in the middle" phenomenon where information in the context

center gets ignored or deprioritized. Therefore, curated, relevant context in smaller windows often outperforms filling larger windows with noise.

What are some common pre-training objectives for LLMs, and how do they work?

Common pre-training objectives for LLMs include masked language modeling (MLM) and autoregressive language modeling. In MLM, random words in a sentence are masked, and the model is trained to predict the masked words based on the surrounding context. This helps the model understand the bidirectional context.

Autoregressive language modeling involves predicting the next word in a sequence and training the model to generate text one token at a time. Both objectives enable the model to learn language patterns and semantics from large corpora, providing a solid foundation for fine-tuning specific tasks.

Looking to get started with Generative AI?

Learn how to work with LLMs in Python right in your browser



What is fine-tuning in the context of LLMs, and why is it important?

[Fine-tuning](#) in the context of LLMs involves taking a pre-trained model and further training it on a smaller, task-specific dataset. This process helps the model adapt its general language understanding to the nuances of the specific application, thereby improving performance.

This is an important technique because it leverages the broad language knowledge acquired during pre-training while modifying the model to perform well on specific applications, such as sentiment analysis, text summarization, or question-answering.

What are some common challenges associated with using LLMs?

Using LLMs comes with several challenges, including:

- **Computational resources:** LLMs require significant computational power and memory, making training and deployment resource-intensive.

- **Bias and fairness:** LLMs can inadvertently learn and propagate biases present in the training data, leading to unfair or biased outputs.
- **Interpretability:** Understanding and explaining the decisions made by LLMs can be difficult due to their complex and opaque nature.
- **Data privacy:** Using large datasets for training can raise concerns about data privacy and security.
- **Cost:** The development, training, and deployment of LLMs can be expensive, limiting their accessibility for smaller organizations.

How do LLMs handle out-of-vocabulary (OOV) words or tokens?

LLMs handle out-of-vocabulary (OOV) words or tokens using techniques like subword tokenization (e.g., Byte Pair Encoding or BPE, and WordPiece). These techniques break down unknown words into smaller, known subword units that the model can process.

This approach ensures that even if a word is not seen during training, the model can still understand and generate text based on its constituent parts, improving flexibility and robustness.

What are embedding layers, and why are they important in LLMs?

Embedding layers are a significant component in LLMs used to convert categorical data, such as words, into dense vector representations. These embeddings capture semantic relationships between words by representing them in a continuous vector space where similar words exhibit stronger proximity. The importance of embedding layers in LLMs includes:

- **Dimensionality reduction:** They reduce the dimensionality of the input data, making it more manageable for the model to process.
- **Semantic understanding:** Embeddings capture nuanced semantic meanings and relationships between words, enhancing the model's ability to understand and generate human-like text.
- **Transfer learning:** Pre-trained embeddings can be used across different models and tasks, providing a solid foundation of language understanding that can be fine-tuned for specific applications.

How do modern LLM positional encodings work, and why did Rotary Position Embeddings (RoPE) become standard?

Positional encodings tell transformers where each token is located in the sequence.

Traditional sinusoidal positional encodings (from Vaswani et al. 2017) used fixed mathematical functions to encode positions statically. Rotary Position Embeddings (RoPE), introduced more recently, have become the

standard in modern LLMs because they offer fundamentally superior properties.

RoPE works by representing positions as rotation angles in complex vector space, rotating token embeddings by an angle proportional to their position. This geometric approach is more efficient and naturally supports interpolation to longer sequences than those seen during training, which is a critical capability for extending context windows. Models like **GPT-5.2** and **Gemini 3** use RoPE as their positional encoding.

Intermediate LLM Interview Questions

Building upon basic concepts, intermediate-level questions delve into the practical techniques used to optimize LLM performance and address challenges related to computational efficiency and model interpretability.

Explain the concept of attention in LLMs and how it is implemented.

The concept of attention in LLMs is a method that allows the model to focus on different parts of the input sequence when making predictions. It dynamically assigns weights to other tokens in the input, highlighting the most relevant ones for the current task.

This is implemented using self-attention, where the model calculates attention scores for each token relative to all other tokens in the sequence, allowing it to capture dependencies regardless of their distance.

bat

no-context vectors

bat



Traditional embedding



Swing the **bat**!

The **bat** flew at night.



Attention embedding



swing

bat

context-aware vectors

bat

flew

The self-attention mechanism is a core component of the Transformer architecture, enabling it to process information efficiently and capture long-range relationships.

What is the role of tokenization in LLM processing?

[Tokenization](#) converts raw text into smaller units called tokens, which can be words, subwords, or characters.

The role of tokenization in LLM processing is vital as it transforms text into a format that the model can understand and process.

Effective tokenization ensures that the model can handle a diverse range of inputs, including rare words and different languages, by breaking them down into manageable pieces. This step is necessary for optimal training and inference, as it standardizes the input and helps the model learn meaningful patterns in the data.

How do you measure the performance of an LLM?

Researchers and practitioners have developed numerous evaluation metrics to gauge the performance of an LLM. Classic metrics include:

- **Perplexity:** Measures how well the model predicts a sample, commonly used in language modeling tasks.
- **Accuracy:** Used for tasks like text classification to measure the proportion of correct predictions.
- **F1 Score:** A harmonic mean of precision and recall, used for tasks like named entity recognition.
- **BLEU (Bilingual Evaluation Understudy) score:** Measures the quality of machine-generated text against reference translations, commonly used in machine translation.
- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation):** A set of metrics that evaluate the overlap between generated text and reference text, often used in summarization tasks. They help quantify the model's effectiveness and guide further improvements.

Beyond traditional metrics, practitioners now use standardized [benchmarks](#) for different purposes, such as [MMLU](#) (57-task knowledge test), [MMMU-Pro](#) (multimodal reasoning), and [HumanEval](#) (code generation). Additionally, leaderboards like [LMArena](#) rank LLMs by human preference. For deployed systems, measuring real-world hallucination rates, latency, and token efficiency has become essential.

What are some techniques for controlling the output of an LLM?

Several techniques can be used to control the output of an LLM, including:

- **Temperature:** Adjusting this parameter during sampling controls the randomness of the output. Lower temperatures produce more deterministic outputs, while higher values return more varied results.
- **Top-K sampling:** Limits the sampling pool to the top K most probable tokens, reducing the likelihood of generating less relevant or nonsensical text.
- **Top-P (nucleus) sampling:** Chooses tokens from the smallest set whose cumulative probability exceeds a threshold P, balancing diversity and coherence.
- **Prompt engineering:** Crafting specific prompts to guide the model towards generating desired outputs by providing context or examples.
- **Control tokens:** Using special tokens to signal the model to generate text in a specific style, format, or content type.

What are some approaches to reduce the computational cost of LLMs?

To reduce the computational cost of LLMs, we can employ:

- **Model pruning:** Removing less important weights or neurons from the model to reduce its size and computational requirements.
- **Quantization:** Converting the model weights from higher precision (e.g., 32-bit floating-point) to lower precision (e.g., 8-bit integer) reduces memory usage and speeds up inference.
- **Distillation:** Training a smaller model (student) to mimic the behavior of a larger, pre-trained model (teacher) to achieve similar performance with fewer resources.
- **Sparse attention:** Using techniques like sparse transformers to limit the attention mechanism to a subset of tokens, reduces computational load.
- **Efficient architectures:** Developing and using efficient model architectures specifically designed to minimize computational demands while maintaining performance, such as the Reformer or Longformer.

What is the importance of model interpretability in LLMs, and how can it be achieved?

Model interpretability is essential for understanding how an LLM makes decisions, which is important for building trust, ensuring accountability, and identifying and mitigating biases. Achieving interpretability can involve different approaches, such as:

- **Attention visualization:** Analyzing attention weights to see which parts of the input the model focuses on.

- **Saliency maps:** Highlighting input features that have the greatest influence on the model's output.
- **Model-Agnostic methods:** Using techniques like LIME (Local Interpretable Model-agnostic Explanations) to explain individual predictions.
- **Layer-wise relevance propagation:** Breaking down the model's predictions into contributions from each layer or neuron.

How do LLMs handle long-term dependencies in text?

LLMs handle long-term dependencies in text through their architecture, particularly the self-attention mechanism, which allows them to consider all tokens in the input sequence simultaneously. This ability to attend to distant tokens helps LLMs capture relationships and dependencies over long contexts.

Additionally, advanced models like the Transformer-XL and Longformer are specifically designed to extend the context window and manage longer sequences more effectively, ensuring better handling of long-term dependencies.

Modern production models use more advanced strategies to handle long-term dependencies and counteract degrading performance with expanding context windows, such as:

- Rotary Position Embeddings (RoPE) for better length extrapolation and superior position encoding.
- Attention patterns optimized for long sequences, including grouped-query and multi-head attention variants.
- Careful knowledge integration via RAG rather than relying solely on context window capacity.

What is Retrieval-Augmented Generation (RAG) and how has it evolved?

RAG combines retrieval mechanisms with generative models to fetch relevant information from external sources during text generation. This approach directly addresses two critical LLM limitations: **hallucination** and knowledge currency. Traditional RAG uses relatively simple retrieval and generation pipelines, but in its evolution to "RAG 2.0", it has become significantly more sophisticated.

Key features of RAG 2.0 include:

- **Recursive retrieval:** Models reason about gaps in retrieved information and proactively perform secondary or tertiary searches to fill knowledge gaps rather than relying on a single retrieval pass
- **Multimodal retrieval:** Integration across diverse formats—text, images, PDFs, videos, and even API calls—enabling richer context gathering

- **Hybrid indexing:** Combined BM25 (keyword-based) and vector (semantic) search to capture both lexical and semantic relevance
- **Re-ranking layers:** Smart filtering of initial retrieval results to reduce noise and irrelevant information
- **Agentic adaptation:** Dynamic strategy selection where the system chooses retrieval approaches based on query complexity and domain

RAG 2.0, combined with LLMs, effectively reduces hallucination rates by 40-60% in production systems compared to base models.

Advanced LLM Interview Questions

Understanding advanced concepts in LLMs is useful for professionals who aim to push the boundaries of what these models can achieve. This section explores complex topics and common challenges faced in the field.

Explain the concept of "few-shot learning" in LLMs and its advantages.

[Few-shot learning](#) in LLMs refers to the model's ability to learn and perform new tasks using only a few examples. This capability leverages the LLM's extensive pre-trained knowledge, enabling it to generalize from a small number of instances.

The primary advantages of few-shot learning include reduced data requirements, as the need for large task-specific datasets is minimized, increased flexibility, allowing the model to adapt to various tasks with minimal fine-tuning, and cost efficiency, as lower data requirements and reduced training times translate to significant cost savings in data collection and computational resources.

What are the differences between autoregressive and masked language models?

Autoregressive and masked language models differ mainly in their prediction approach and task suitability. Autoregressive models, like [GPT-5.2](#), [Claude 4.5 Opus](#), and [Gemini 3](#), predict the next word in a sequence based on the preceding words, generating text one token at a time.

These models are particularly well-suited for text-generation tasks. In contrast, masked language models, such as [BERT](#), randomly mask words in a sentence and train the model to predict these masked words based on the surrounding context. This bidirectional approach helps the model understand context from both directions, making it ideal for text classification and question-answering tasks.

How can you incorporate external knowledge into an LLM?

Incorporating external knowledge into an LLM can be achieved through several methods:

- **Knowledge graph integration:** Augmenting the model's input with information from structured knowledge graphs to provide contextual information.
- **Retrieval-Augmented Generation (RAG):** Combines retrieval methods with generative models to fetch relevant information from external sources during text generation. Modern RAG 2.0 systems use recursive retrieval, hybrid indexing, and re-ranking for superior knowledge integration compared to simple retrieval approaches.
- **Fine-tuning with domain-specific data:** Training the model on additional datasets that contain the required knowledge to specialize it for specific tasks or domains.
- **Prompt engineering:** Designing prompts that guide the model to utilize external knowledge effectively during inference.

What are some challenges associated with deploying LLMs in production?

Deploying LLMs in production involves various challenges:

- **Scalability:** Ensuring the model can handle large volumes of requests efficiently often requires significant computational resources and optimized infrastructure.
- **Latency:** Minimizing the response time to provide real-time or near-real-time outputs is critical for applications like chatbots and virtual assistants.
- **Monitoring and maintenance:** Continuously monitoring model performance and updating it to handle evolving data and tasks requires robust monitoring systems and regular updates.
- **Ethical and legal considerations:** Addressing issues related to bias, privacy, and compliance with regulations is essential to avoid ethical pitfalls and legal repercussions.
- **Resource management:** Managing the significant computational resources required for inference ensures cost-effectiveness and involves optimizing hardware and software configurations.

How do you handle model degradation over time in deployed LLMs?

Model degradation occurs when the performance of an LLM declines over time due to changes in the underlying data distribution. Handling model degradation involves regular retraining with updated data to maintain performance. Continuous monitoring is necessary to track the model's performance and detect signs of degradation.

Incremental learning techniques allow the model to learn from new data without forgetting previously learned information. Additionally, A/B testing compares the current model's performance with new versions

and helps identify potential improvements before full deployment.

What are some techniques to ensure the ethical use of LLMs?

To ensure the [ethical use of LLMs](#), several techniques can be implemented:

- **Bias mitigation:** Applying strategies to identify and reduce biases in training data and model outputs, such as using balanced datasets and bias detection tools.
- **Transparency and explainability:** Developing models that provide interpretable and explainable outputs to foster trust and accountability, including using attention visualization and saliency maps.
- **User Consent and privacy:** Ensuring data used for training and inference complies with privacy regulations and obtaining user consent where necessary.
- **Fairness audits:** Conduct regular audits to evaluate the fairness and ethical implications of the model's behavior.
- **Responsible deployment:** Setting guidelines and policies for [responsible AI](#) deployment, including handling harmful or inappropriate content generated by the model.

How can you ensure the security of data used with LLMs?

Securing data used with LLMs requires implementing various measures. These include using encryption techniques for data at rest and in transit to protect against unauthorized access. Strict access controls are necessary to ensure that only authorized personnel can access sensitive data.

Anonymizing data to remove personally identifiable information (PII) before using it for training or inference is also crucial. Additionally, compliance with data protection regulations like GDPR or CCPA is essential to avoid legal issues.

These measures help protect data integrity, confidentiality, and availability. This protection is critical for maintaining user trust and adhering to regulatory standards.

How does reinforcement learning from human feedback (RLHF) improve LLM output quality and safety, and how does it compare to newer alignment methods like DPO and RLAIIF?

RLHF is a technique that involves training an LLM to align its outputs with human preferences by incorporating feedback from human evaluators. This iterative process helps the model learn to generate responses that are not only accurate but also safe, unbiased, and helpful.

However, RLHF comes with challenges. One challenge is the potential for bias in the human feedback, as different evaluators might have varying preferences and interpretations.

Another challenge is the scalability of the feedback process, as collecting and incorporating large amounts of human feedback can be time-consuming and expensive. Additionally, ensuring that the reward model used in RLHF accurately captures the desired behaviors and values can be tricky. The PPO (Proximal Policy Optimization) optimization step adds complexity and can introduce instability during training.

Modern alternatives have emerged that address many of RLHF's limitations:

- **DPO (Direct Preference Optimization):** Eliminates the separate reward model entirely, using implicit reward signals derived directly from preference pairs. This approach is simpler, more stable, and computationally efficient than PPO-based RLHF. DPO is now widely adopted by major AI labs as an alternative or complement to traditional RLHF.
- **GRPO (Gradient-based Reward Policy Optimization):** A variant that combines the benefits of DPO's simplicity with aspects of PPO-style learning, offering middle-ground trade-offs.
- **RLAIF (Reinforcement Learning from AI Feedback):** Replaces human evaluators with AI feedback (such as ratings from stronger models or specialized evaluators). This approach dramatically reduces scalability challenges and costs since generating AI feedback is cheaper and faster than collecting human feedback.

Most production models use DPO, RLAIF, or hybrid combinations rather than pure PPO-based RLHF.

Compare state-space models (like Mamba) with Transformers. What are the trade-offs, and when would you use each?

Knowledge-based hallucinations happen when the model doesn't have the right facts (or has outdated facts), so it invents plausible-sounding information. Fixes typically involve:

- Grounding the model with retrieval (RAG)
- Adding verified sources
- Domain-specific fine-tuning

Logic-based hallucinations happen when the model has relevant information but reasons incorrectly or produces inconsistencies. Common fixes are:

- Structured prompting
- Verification/self-check steps (often via agentic workflows)
- Targeted evaluation

What is the difference between knowledge-based and logic-based hallucinations in LLMs, and how do you address each?

While Transformers dominate LLMs today, state-space models (SSMs) like Mamba have emerged as a competitive architectural paradigm with fundamentally different computational and performance properties. Understanding both is increasingly important for modern LLM professionals.

Transformers:

- **Strengths:** Excellent at information retrieval, copy/paste tasks, and in-context learning. Proven architecture at scale with extensive tooling and infrastructure. Strong empirical performance on code generation and diverse benchmarks.
- **Weaknesses:** $O(N^2)$ attention complexity creates quadratic memory and computational requirements. This manifests as performance degradation on very long sequences (>100K tokens) and high latency for long-context applications.
- **Use cases and recommendations:** Most production LLMs today (GPT, Claude, Gemini, Llama). Recommended for current real-world applications and the foreseeable future.

State-Space Models (Mamba, Mamba-2):

- **Strengths:** $O(N)$ complexity with linear memory scaling. 5-10x faster inference on long sequences (>50K tokens). Better theoretical foundations for signal processing. More predictable scaling behavior.
- **Weaknesses:** Require 10x more training data than Transformers to achieve comparable performance. Weaker on information retrieval and copying tasks. Less mature ecosystem with fewer tools, libraries, and trained models.
- **Use cases and recommendations:** Emerging for specialized applications—long-document processing, real-time systems with latency constraints, streaming applications, and potentially specialized domains where $O(N)$ scaling is critical.

Recent research demonstrates that Transformers and SSMs are complementary rather than directly competitive, with different use cases and constraints favoring different architectures. Transformers remain the practical choice for general-purpose LLMs and will likely continue dominating for now. A hybrid future is likely, with specialized models for different computational and task requirements.

LLM Interview Questions for Prompt Engineers

[Prompt engineering](#) is an important aspect of utilizing LLMs. It involves crafting precise and effective prompts to generate desired responses from the model. This section examines key questions that prompt engineers may encounter.

What is prompt engineering, and why is it crucial for working with LLMs?

Prompt engineering involves designing and refining prompts to guide LLMs in generating accurate and relevant outputs. It's vital for working with LLMs because the quality of the prompt directly impacts the model's performance.

Effective prompts can enhance the model's ability to understand the task, generate accurate and relevant responses, and reduce the likelihood of errors.

Prompt engineering is essential for maximizing the utility of LLMs in various applications, from text generation to complex problem-solving tasks.

Can you provide examples of different prompting techniques (zero-shot, few-shot, chain-of-thought) and explain when to use them?

- **Zero-shot prompting:** Provides the model with a task description without any examples. Typically used when there are no available examples or when we want to test the model's general understanding and flexibility.
- **Few-shot prompting:** Supplies a few examples along with the task description to guide the model. This is useful when the model needs context or examples to better understand the task.
- **Chain-of-Thought prompting:** Breaks down a complex task into smaller, sequential steps that the model can follow. This can be beneficial for tasks that require logical reasoning and multi-step problem-solving.
- **Agentic prompting:** Structure prompts to make models act as autonomous agents that call tools/APIs, collect information, and reason across multiple steps. Prompts include action space definitions, tool descriptions, and explicit reasoning requirements.

How do you evaluate the effectiveness of a prompt?

Evaluating the effectiveness of a prompt involves:

- **Output quality:** Assessing the relevance, coherence, and accuracy of the model's responses.
- **Consistency:** Checking if the model consistently produces high-quality outputs across different inputs.
- **Task-specific metrics:** Using task-specific evaluation metrics, such as BLEU for translation or ROUGE for summarization, to measure performance.
- **Human evaluation:** Involving human reviewers to provide qualitative feedback on the model's outputs.

- **A/B testing:** Comparing different prompts to determine which one yields better performance.

What are some strategies for avoiding common pitfalls in prompt design (e.g., leading questions, ambiguous instructions)?

- **Avoid leading questions:** Ensure that prompts do not imply a specific answer, which can bias the model's response.
- **Clear and concise instructions:** Provide unambiguous and straightforward instructions to reduce confusion.
- **Context provision:** Include relevant context to help the model understand the task without overloading it with unnecessary information.
- **Iterative testing:** Continuously test and refine prompts based on the model's outputs and performance.

How do you approach iterative prompt refinement to improve LLM performance?

Iterative prompt refinement involves:

- **Initial design:** Start with a basic prompt based on task requirements.
- **Testing and evaluation:** Assess the prompt's performance using various metrics and obtain feedback.
- **Analysis:** Identify weaknesses or areas for improvement in the prompt.
- **Refinement:** Make adjustments to the prompt to address identified issues.
- **Repeat:** Repeat the testing and refinement process until the desired performance is achieved.

What tools or frameworks do you use to streamline the prompt engineering process?

Several tools and frameworks can streamline the prompt engineering process:

- **LLM-specific platforms:** LangChain and LlamaIndex for prompt chaining and building agentic systems; Vercel AI SDK for rapid prototyping and deployment.
- **Prompt testing & optimization:** PromptFoo (automated prompt testing across scenarios), DSPy (programmatic prompt optimization), Haystack (RAG pipeline construction).

- **Interactive development environments (IDEs):** Jupyter Notebook for experimentation, VS Code with LLM extensions (GitHub Copilot, Cursor), Streamlit for building interactive UI prototypes.
- **APIs and SDKs:** OpenAI API, Anthropic SDK, Together AI, and LM Studio for local model deployment.
- **Version control & collaboration:** Git-based prompt versioning, Weights & Biases for experiment tracking and comparison, HuggingFace Hub for sharing models and prompts.
- **Agentic frameworks:** AutoGPT, OpenAI Assistants API, LangGraph for building complex multi-step agentic workflows.
- **Evaluation & monitoring:** Ragas (RAG evaluation metrics), LangSmith (tracing, debugging, and production monitoring), Braintrust (end-to-end LLM observability).

How do you handle challenges like hallucination or bias in LLM outputs through prompt engineering?

This question addresses the ethical and practical issues of LLM-generated content. A strong answer would demonstrate awareness of these problems and discuss techniques like the following.

Hallucination mitigation techniques:

- **Fact verification prompts:** Incorporate prompts that encourage the model to verify its information against reliable sources and explicitly cite evidence.
- **Retrieval integration:** Use RAG or RAG 2.0 systems to ground responses in actual documents, reducing invented facts by requiring the model to reference retrieved information.
- **Agentic verification loops:** Structure prompts to make the model perform multi-step verification, where it checks its own work before responding—essentially creating self-verification agents.
- **Constraint-based prompts:** Define output format and logical constraints that make inconsistencies or unsupported claims apparent.

Bias mitigation techniques:

- **Diverse perspective prompts:** Guide the model to consider multiple viewpoints and stakeholder perspectives before concluding.
- **Bias detection prompts:** Ask the model to identify potential biases in its own reasoning and output.

- **Counterfactual prompts:** Request alternative scenarios or perspectives to challenge initial assumptions and test reasoning robustness.
- **Instruction clarity:** Explicitly specify that responses should be balanced, acknowledge limitations, and avoid discriminatory language.

Can you explain the role of prompt templates and how they are used in prompt engineering?

Prompt templates provide a structured format for prompts, often including placeholders for specific information or instructions. They can be reused across different tasks and scenarios, improving consistency and efficiency in prompt design.

A good answer would explain how prompt templates can be used to encapsulate best practices, incorporate domain-specific knowledge, and streamline the process of generating effective prompts for various applications.

How does the choice of a tokenizer impact prompt engineering and model performance?

The tokenizer plays a crucial role in how the LLM interprets and processes the input prompt. Different tokenizers have varying vocabulary sizes and handle out-of-vocabulary (OOV) words differently. A subword tokenizer like Byte Pair Encoding (BPE) can handle OOV words by breaking them into smaller subword units, while a word-based tokenizer might treat them as unknown tokens.

The choice of tokenizer can impact model performance in several ways. For instance, a subword tokenizer might be more effective in capturing the meaning of rare or technical terms, while a word-based tokenizer might be simpler and faster for general-purpose language tasks.

In prompt engineering, the choice of tokenizer can influence how you structure your prompts. For example, if you're using a subword tokenizer, you might need to pay more attention to how words are split into subwords to ensure that the model captures the intended meaning.

What is agentic prompting, and how does it differ from traditional prompt engineering?

Agentic prompting is a prompt design approach where the model is instructed to act like an “agent” that can plan, take actions (for example, call tools/APIs or retrieve documents), observe the results, and iterate until it completes an objective.

Unlike traditional prompt engineering, which mainly tries to get the best possible response in one shot through clear instructions, examples, and formatting constraints, agentic prompting emphasizes a multi-step control loop (plan → act → observe → refine).

It's especially useful for tasks where correctness depends on interacting with external systems (databases, search, code execution) or verifying intermediate results rather than relying on the model's internal knowledge alone.

Good agentic prompts usually define the objective, available tools/actions, how to decide which action to take, and when to stop. It's also common to add explicit verification steps (e.g., "check retrieved sources" or "recompute before final answer") to reduce hallucinations and improve reliability.